For advanced bean configuration, we can use XML and Java. XML based configuration is generally not recommended, as it can grow big and unreadable really quick, but then again this would depend on preferences.

For this article, we'll use the Java based configuration for a sample Redis application. The *@Configuration* annotation allows for a class to be loaded as a configuration object. This configures a library called Jedis to be used with the given server host and port. The *@Value* annotation can be used to read values from the environment's *properties* file.

The *@Bean* annotation says that a JedisConnectionFactory has to be treated as a Spring Bean and is instantiated by the method:

```
@Configuration
class RedisConfiguration {

    @Value("${redis.server.host}")
    private String server;


    @Value("${redis.server.port}")
    private Integer port;

    @Bean
    public JedisConnectionFactory
redisConnectionFactory() {
        RedisStandaloneConfiguration config = new RedisSta
ndaloneConfiguration(server, port);
        return new JedisConnectionFactory(config);
    }
}
```

Exception handling: Applications have errors and exceptional cases that may not always be possible to be handled by developers. If an issue occurs, we'd like it to pop up and inform the user that something has gone wrong without divulging too much information about the system. Spring has a number of options for dealing with this.

Prior to Spring 3.2, these were bundled with HandlerExceptionResolver and *@ExceptionHandler;* however, both of them had downsides of usability.

The *@ExceptionHandler* annotation may be added to a controller method, but it can only handle exceptions for that particular controller. As a result, all exceptions in all controllers have to be handled individually. The HandlerExceptionResolver is modular and functional, but it still relies on the older ModelView design and isn't a popular solution for REST apps that don't have views developed.

From Spring 3.2, we have the *@ControllerAdvice* annotation that allows us to easily handle all exceptions in a RESTful way. This can be defined once and will work globally for all exceptions in all controllers.

```
@ControllerAdvice
public class RestResponseEntityExceptionHandler
  extends ResponseEntityExceptionHandler {


    @ExceptionHandler(Exception.class)
    protected ResponseEntity<Object> handleError(
      RuntimeException ex, WebRequest request) {
        String bodyOfResponse = "Something went wrong";
        return handleExceptionInternal(ex, bodyOfResponse,
          new HttpHeaders(), HttpStatus.ERROR, request);
    }

}
```

We can also mention specific exceptions to be handled by specific handlers; for example, we can send a *409 CONFLICT* response for a DataIntegrityViolation exception by specifying a value parameter in the *ExceptionHandler* annotation.

```
@ResponseStatus(value=HttpStatus.CONFLICT,
                 reason="Data integrity violation")
  @ExceptionHandler(DataIntegrityViolationException.class)
  public void conflict() {
     // log it

  }
```

In this article, we looked at how Spring operates and understood how the code is structured. Spring's capabilities can be leveraged to build very powerful applications without writing too much boilerplate.

There is more to study about Spring, and understanding its intricacies can help make the best out of what Spring has to offer. More can be found at Spring's extensive official documentation at *https://docs.spring.io/spring-framework/docs/current/reference/html/*. END

By: Dedipyaman Das

The author is a software engineer and works at Bajaj Finserv Health Ltd.